

C++ Identifier Syntax using Unicode Standard Annex 31

Document #: D1949R0
Date: 2019-11-03
Project: Programming Language C++
SG16
EWG
CWG
Reply-to: Steve Downey
<sdowney@gmail.com, sdowney2@bloomberg.net>

§ 1 Abstract

In response to NL 029 : Disallow zero-width and control characters

Adopt Unicode Annex 31 as part of C++ 23. - That C++ identifiers match the pattern (XID_START + _) + XID_CONTINUE*. - That portable source is required to be normalized as NFC. - That using unassigned code points ill-formed.

§ 2 Poll before discussion

The current state, allowing control characters, ZWJ, and unassigned codepoints in C++ identifiers is not a defect, and is working as designed, and does not need to be addressed

§ 3 Addressing identifiers in a more principled ways

[UNICODE IDENTIFIER AND PATTERN SYNTAX](#) is an attempt to provide a normative way of specifying definitions of general-purpose identifiers for use in programming languages. It has evolved significantly over the years, in particular since the time that C++ 11 was specified. In particular, the characters that were allowed as identifiers, and the patterns, were not stable at the time of C++11, which is the last time identifiers were addressed in the standard. In addition, at that time, ISO was promulgating advice suggesting a list of code points as the recommended method for ISO standards to specify identifiers.

Today the definitions in UAX31 can be used to provide stable definitions for programming language identifiers, with guarantees that an identifier will not be

invalidated by later standards.

Originally, UAX31 relied on derived properties of characters, ID_START and ID_CONTINUE, however those properties relied on fundamental properties that could change over time. The unicode database now provides XID_START and XID_CONTINUE, based on the same characteristics, but with an additional stability guarantee. The Unicode database now provides explicit classification of both.

The original definitions closely match the identifier syntax of C:

Properties	General Description of Coverage
ID_Start	ID_Start characters are derived from the Unicode General_Category of uppercase letters, lowercase letters, titlecase letters, modifier letters, other letters, letter numbers, plus Other_ID_Start, minus Pattern_Syntax and Pattern_White_Space code points.
	In set notation:
	<code>[p{L}p{Nl}-p{Pattern_Syntax}-p{Pattern_White_Space}]</code>
ID_Continue	ID_Continue characters include ID_Start characters, plus characters having the Unicode General_Category of nonspacing marks, spacing combining marks, decimal number, connector punctuation, plus Other_ID_Continue , minus Pattern_Syntax and Pattern_White_Space code points.
	In set notation:
	<code>[p{ID_Start}p{Mc}p{Pc}p{Other_ID_Continue}-p{Pattern_Syntax}-p{Pattern_White_Space}]</code>

The X versions of the properties start the same, but are guaranteed stable in subsequent Unicode standards

§ 4 Issues

- Continue does not include ZWJ, which some scripts require
- Does not exclude homoglyph attack
- Does not require the compiler to normalize identifiers
- Does not allow emoji

§ 5 History

Using an explicit list of Unicode characters was considered a best practice for ISO standardization in TR 10176:2003 Guidelines for the preparation of programming language standards.

National body comment CA 24 for C++11:

A list of issues related TR 10176:2003:

- “Combining characters should not appear as the first character of an identifier.” Reference: ISO/IEC TR 10176:2003 (Annex A) This is not reflected in FCD.
- Restrictions on the first character of an identifier are not observed as recommended in TR 10176:2003. The inclusion of digits (outside of those in the basic character set) under identifier-nondigit is implied by FCD.
- It is implied that only the “main listing” from Annex A is included for C++. That is, the list ends with the Special Characters section. This is not made explicit in FCD. Existing practice in C++03 as well as WG 14 (C, as of N1425) and WG 4 (COBOL, as of N4315) is to include a list in a normative Annex.
- Specify width sensitivity as implied by C++03: is not the same as A. Case sensitivity is already stated in [lex.name].

N3146 in 2010-10-04 considered using UAX31, but at the time there were stability issues with identifiers, and came down on the side of explicit white listing.

The Unicode standard has since made stability guarantees about identifiers, and created the `XID_START` and `XID_CONTINUE` properties to alleviate the stability concerns that existed in 2010.

§ 6 Wording

Wording to follow based on SG16 and EWG guidance. There is much prior art to follow based on similar proposals and adoption in Rust and Swift.

Explicit universal character names and codepoints are available for particular Unicode standards from the published database, and could be appended as an appendix.